Código Intermedio

Ing. Mario Ramírez Tobar

Subprogramas



DEFINICIONES S-ATRIBUIDAS



¿Que son?

Los subprogramas son un conjunto de instrucciones aislado del programa principal.

Características

- Tienen nombre
- Tienen un cuerpo con instrucciones
- Pueden devolver un valor (funciones)
- Se utiliza el nombre para llamar al subprograma
- Pueden tener entradas (parámetros)
- Los parámetros tienen tipo definido
- Los valores de entrada son los argumentos.

Ejemplo

```
function factorial( x : integer ) : integer;
        var y , z : integer ;
begin
    y := 1 ;
    for z := 1 to x do y := y * z ;
    y := suma(y, z)
    factorial := y
end ;
```

Características

- Los subprogramas alteran el flujo de ejecución del programa

```
begin
    x := 5 ;
    x := factorial( x ) ;
    for x := 0 to 7 do v[ x ] := x ;
    v := suma( v , x )
end
```

Características

- Cuando se activa un subprograma (invocación o llamada) se crea un espacio en memoria (stack o pila) dedicado a dicha activación.
- Cuando se finaliza la activación de un subprograma se elimina su pila.
- Si el subprograma es una función, entonces existe una posición en su pila que almacena el valor a retornar

Ejemplo

x	5
x	5
y	?
z	?
factorial	120

```
begin
    x := 5 ;
    x := factorial( x ) ;
    for x := 0 to 7 do v[ x ] := x ;
    v := suma( v , x )
end
```

Características

- Justo antes de terminar la activación de una función se tiene acceso a la posición de la pila con el valor de retorno.
- Al finalizar la activación, el control del flujo regresa al punto donde se hizo la activación.

¿Cómo llevamos el control?

Para llevar el control de las activaciones y generar el código simple que las represente, podemos usar un árbol de activaciones.

Sobrecarga

Un subprograma puede comportarse de diferentes maneras dependiendo de los parámetros.

int suma(int a, int b)

Int suma(String a, String b)

Errores

Aquí hay un error de compilación, por qué?

```
int suma(int a, int b)
float suma(int a, int b)
```

Paso de parámetros

Por valor: se copia el valor al stack del subprograma a activar.

Por referencia: se copia la dirección de memoria que corresponde a la variable. La dirección de memoria se usa para leer/escribir valores del parámetro.

¿Porque nos interesa?

Nos interesa recordar todas esas características ya que el compilador se encarga de generar el código que representa el flujo de control de un programa.

El código final o máquina no tiene estructuras de control, sino que es código secuencial.

Código Intermedio





¿Porque es importante?

El lenguaje de alto nivel analizado debe transformarse parcialmente a una representación intermedia que es más sencilla para que se tenga una similitud con el código final.

¿Qué es?

La representación intermedia puede ser código o también estructuras intermedias que poseen toda la información para describir una operación de alto nivel.

Una representación intermedia es el código 3 direcciones

Código 3 direcciones





¿Qué es?

El código 3 direcciones propone que una instrucción no puede acceder a más de 3 direcciones en memoria al mismo tiempo. En tiempo de ejecución solo accedemos a posiciones en memoria.

¿Que no se permite?

Esto no es permitido en codigo 3D

```
a = 3 + a + val + 25 / 2;
```

Estamos accediendo a 6 posiciones en memoria en una sola instrucción.

¿Que se permite?

1. dest = op1 op op2

2. dest = val

3. dest = op op1

Ejemplos

```
a = 25 + 5
b = a * 50

c = 25
a = b

a = -25
c = !algo
```

Hay que transformar

x = 25 + y * z + (5 - x)

C3D:

t1 = y * z

t2 = 25 + t1

t3 = 5 - x

t4 = t2 + t3

x = t4

¿Que se permite?

4. etq et1
5. goto et1
6. if x goto et
7. if op1 oprel op2 goto et
8. ifFalse x goto et

Transformemos de nuevo

```
x = 25 + 12 + a;
z = -x * 5 + 2;
if (x > z) {
    y = z + 25 + 1;
} else if (a > b) {
    a = z + 1 * x;
} else {
    z = 5 * 1 + c;
}
```

x = 25 + 12 + a; **z = -x * 5 + 2;**	t1 = 25 + 12 t2 = t1 + a x = t2 t3 = -x t4 = t3 * 5 t5 = t4 + 2 z = t5

if (x > z) {
  y = z + 25 + 1;
} else if (a > b){
  a = z + 1 * x;
} else {
  z = 5 * 1 + c;
}
	If x > z goto et1
	goto et2
	et1:
	t6 = z+ 25
	t7 = t6 + 1
	y = t7
	goto et3
	et2:
	t8 = 1 * x
	t9 = z + t8
	a = t9
	// aquí puede ser opcional: goto et3

```
if (x > z) {
        y = z + 25 + 1;
} else if (a > b){
        a = z + 1 * x;
} else {
        z = 5 * 1 + c;
}
```

```
et3:
if x > z goto et1
goto et2
et1:
t1 = z + 25
t2 = t1 + 1
y = t2
goto etF
et2:
if a > b goto et3
goto et4
et3:
t3 = 1 * x
t4 = z + t3
a = t4
goto etF
et4:
t5 = 5 * 1
t6 = t5 + c
z = t6
etF:
//resto del código
```

Otro Ejemplo

do {
a++;
} while (z < x);
	et1:
	t1 = a + 1
	a = t1
	If z < x goto et1
	goto et2:
	et2:
	// resto del código

Ejercicio

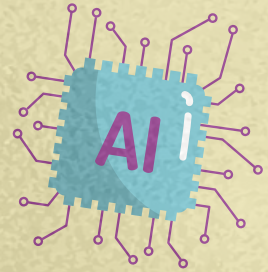




```
for (i = 0; i < 10; i = i * 2) {
    a++;
}
```

```
for (i = 0; i < 10; i = i * 2) {
    a++;
}
```

```
i = 0;
et1:
if i < 10 goto et2
goto et3
Et2:
// instrucciones
t1 = a + 1
a = t1
// codigo incremental
t2 = i * 2
i = t2
goto et1
et3:
// otras instrucciones
```



Arreglos



¿Que són?

Los arreglos son estructuras estáticas que permiten almacenar valores de un mismo tipo.

No es más que un conjunto de espacios consecutivos en la memoria.

Ejemplo

int a[] = new int[5]; a[5] = 12;	t1 = 12 a[5] = t1 // solo se usan hasta 3 direcciones
a[c + 1] = 15 + 1 + b;	t1 = 15 + 1 t2 = t1 + b t3 = c + 1 a[t3] = t2 // solo se usan hasta 3 direcciones Dependiendo de la implementación el resultado también puede ser: t1 = c + 1 t2 = 15 + 1 t3 = t2 + b a[t1] = t3

a[c + 1] = a[15 + 1 + b] + 5 * 2;	t1 = 15 + 1 t2 = t1 + b t3 = a[t2] t4 = 5 * 2 t5 = t3 + t4 t6 = c + 1 a[t6] = t5

¿Multidimensional?

No existe

Los arreglos de múltiples dimensiones no existen en código intermedio.

En código intermedio deben ‘aplanarse’ las dimensiones en una sola.

Int a[][] = new int[2][4]

En total son 8 posiciones:

0,0	0,1	0,2	0,3
1,0	1,1	1,2	1,3

0,0	0,1	0,2	0,3	1,0	1,1	1,2	1,3

¿Que se asume?

Se asume que solo se tiene una dimensión y se calculan los nuevos índices del arreglo aplanado en base al índice de cada dimensión y al tamaño de las dimensiones:

Para dos dimensiones:
Posicion = indiceA * dimensionB + indiceB

Ejemplo

Int a[][] = new int[2][4]

Para esta posición

a[1][3]

La nueva posición es: 1 * 4 + 3 = 7

a[1][0]

La nueva posición es: 1 * 4 + 0 = 4

0,0	0,1	0,2	0,3
1,0	1,1	1,2	1,3

0,0 0	0,1 1	0,2 2	0,3 3	1,0 4	1,1 5	1,2 6	1,3 7

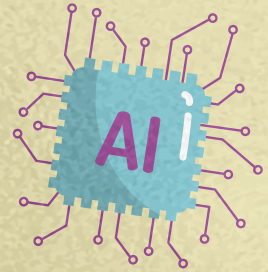
Ejemplo

int a[][] = new int[2][4];	t1 = 1 * 4
a[1][3] = 25;	t2 = t1 + 3
	a[t2] = 25
a[1][0] = 25;	t3 = 1 * 4
	t4 = t3 + 0
	a[t4] = 25
a[1 + 5][c + 1] = 5	t1 = 1 + 5 // indice A
	t2 = c + 1 // indice B
	t3 = t1 * 4
	t4 = t3 + t2
	a[t4] = 5

Ejemplo

Para tres dimensiones:

```
Posicion =
        (indiceA * dimensionB * dimensionC)
        + (indiceB * dimensionC)
        + indiceC
```



Expresiones booleanas



¿Que són?

Estas expresiones involucran operadores relacionales y operadores lógicos como AND OR NOT.

Estas expresiones se pueden asignar a variables o se usan en condicionales.

Las variables booleanas sólo almacenan 1 o 0

Ejemplo

```
boolean x = 25 > 0 OR 30 == id1
if (x > 25 AND NOT(30 > id2)) {
    // código
}
```

Jerarquías

1. NOT (!) tiene la mayor jerarquía.
2. AND (&&) actúa como una multiplicación lógica (conjunción).
3. OR (||) actúa como una suma lógica (disyunción) y tiene la menor jerarquía.

Por eso, el compilador agrupa primero los bloques unidos por AND (las multiplicaciones) antes de unirlos mediante el OR (la suma). El NOT se aplica de inmediato a lo que tiene justo a su derecha (o al paréntesis que le sigue), al igual que un signo negativo en álgebra.

¿Cómo se logra?

Las expresiones booleanas en el TAC (Three-Address Code) funcionan transformando operaciones lógicas complejas en saltos condicionales y asignaciones simples usando etiquetas y variables temporales.

Cuando traducimos un AST a código de tres direcciones, cada nodo booleano no "devuelve" un valor; en su lugar, recibe desde arriba dos etiquetas:

- B.verdadero: A dónde saltar si la condición es verdadera.
- B.falso: A dónde saltar si la condición es falsa.

Reglas

Efecto del NOT (!X): Intercambian las etiquetas que le pasa a su hijo **X**, y no genera ninguna instrucción TAC.

Efecto del AND (X && Y): Para que sea verdadero, ambos deben serlo, Y se hace lo siguiente:

1. Al hijo **X** se le da una etiqueta nueva (*etiqueta_siguiente*) para su salida verdadera, y el mismo falso del padre. Si **X** es verdadero, el flujo avanza a evaluar **Y**.
2. Al hijo **Y** se le dan las etiquetas originales del padre.

Efecto del OR (X || Y): Basta con que uno sea verdadero y se hace lo siguiente:

1. Al hijo **X** se le da el verdadero del padre, y una (*etiqueta_siguiente*) para su salida falsa. Si **X** falla, el flujo avanza a evaluar **Y**.
2. Al hijo **Y** se le dan las etiquetas originales del padre.

```
boolean x = false;
x = b > 25
```

```
x = 0
if b > 25 goto et1
goto et2
et1:
x = 1
goto et3
et2:
x = 0
et3:
```

```
x = z + 1 == 25;
```

```
t1 = z + 1
if t1 == 25 goto et1
goto et2
et1:
x = 1
goto et3
et2:
x = 0
et3:
```

boolean x = 25 > 0 OR 30 == id1	if 25 > 0 goto et1 goto et2 et1: x = 1 goto et3 et2: if 30 == id1 goto et1 goto et4 et4: x = 0 et3:

boolean x = 25 > 0 OR 30 == id1	ifFalse 25 > 0 goto et1 goto et2 et1: if 30 == id1 goto et3 goto et4 et2: et3: x = 1 goto etSalida et4: x = 0 etSalida:

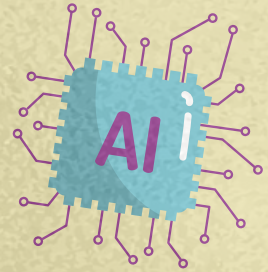
boolean x = 25 > 0 AND 30 == id1	if 25 > 0 goto et1 goto et2 et1: if 30 == id1 goto et3 goto et4: et3: x = 1 goto etSalida et2: et4: x = 0 etSalida:

boolean x = NOT(30 == id1)	if 30 == id1 goto et1 goto et2 et1: x = 0 goto et3 et2: x = 1 et3:

boolean x = not(25 > x AND z == 1 OR a > 5);	if 25 > x goto et1 goto et2 et1: if z == 1 goto et3 goto et4 et3: x = 0 goto etSalida et4: et2: if a > 5 goto et5 goto et6 et5: x = 0 goto etSalida et6: x = 1 goto etSalida etSalida

```
if ( not(25 > x AND z == 1 OR a > 5) ) {
    CODE
}
```

```
if 25 > x goto et1
goto et2
et1:
        if z == 1 goto etCode
        goto et3
et2:    et3:
        if a > 5 goto et4
        goto et5
        et4:
                goto etCode
        et5:
                goto etSalida
etCode:
        CODE
        goto etSalida
etSalida:
        SALIDA
```



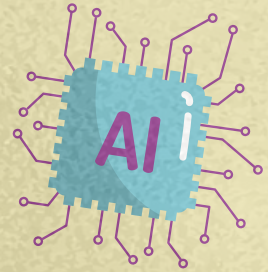
Llamadas a funciones



a = b * unaFuncion(x + 1, z * 25 / 2, 5);	t1 = x + 1 t2 = z * 25 t3 = t2 / 2 t4 = 5 param t1 param t3 param t4 t5 = call unaFuncion 3 t6 = b * t5 a = t6

```
a = b * unaFuncion(otraFuncion(x + 1, b), z * 25 / 2, 5);
```

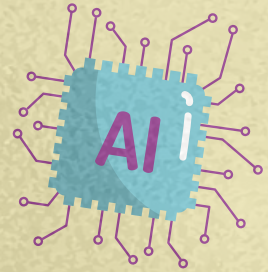
```
t1 = x + 1
param t1
param b
t2 = call otraFuncion 2
t3 = z * 25
t4 = t3 / 2
t5 = 5
param t2
param t4
param t5
t6 = call unaFuncion 3
t7 = b * t6
a = t7
```



Impresión a consola



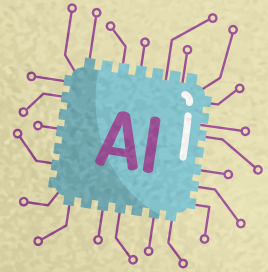
System.out.print("Hola Mundo " + id1 + "aaa");	t1 = concat "Hola Mundo " id1 t2 = concat t1 "aaaa" print t2
System.out.print("Hola Mundo " + id1 + "aaa");	t1 = "Hola Mundo " t2 = id1 t3 = "aaa" print t1 print t2 print t3
System.out.println("Hola Mundo " + id1 + "aaa");	t1 = concat "Hola Mundo " id1 t2 = concat t1 "aaaa" t3 = t2 + SALTO print t3



Lectura desde consola



scanf("%d", id); scanner.nextLine();	SCAN_INT id SCAN_FLOAT id SCAN_CHAR id SCAN_TEXT id



Constantes



Reglas

1. Las constantes son valores que se almacenan en un área específica de la memoria, al inicio.

2. Los valores se deben poder calcular al momento de cargar la memoria, por lo mismo es que las constantes no pueden depender de variables, sino más bien del valor de otras constantes y literales.

Const int PI = 3.1416; Const DATA = 5; Const TEXT = “Hola ” + 5 + “ Mundo” ; Const OTRA = PI * DATA + 1;	PI = 3.1416 DATA = 5 t1 = concat “Hola “ 5 t2 = concat t1 “ Mundo” TEXT = t2 t3 = PI * DATA t4 = t3 + 1 OTRA = t4

CUARTETOS

¿Qué son?

Son una representación estructural del codigo 3 direcciones.

Tienen 4 elementos:
Operador, operando1, operando2, destino y es otra representacion intermedia del codigo de alto nivel.
La intención es generar cuartetos y posterior a ello generar código 3 direcciones o código 3 direcciones ejecutable a partir de los cuartetos.

```
t1 =  enos c
t2 = b * t1
t3 =  enos c
t4 = b * t3
t5 = t2 + t4
 a = t5
```

(a) Código de tres direcciones

	op	arg_1	arg_2	*resultado*
0	enos	c		t_1
1	*	b	t_1	t_2
2	enos	c		t_3
3	*	b	t_3	t_4
4	+	t_2	t_4	t_5
5	=	t_5		a
	...			

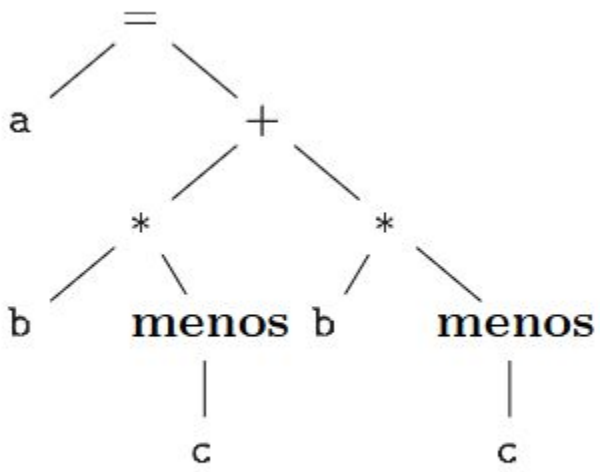
(b) Cuádruplos

Tripletas

¿Qué son?

Son una representación estructural del codigo 3 direcciones.

Tienen 3 elementos:
Operador, operando1, operando2 y es otra representacion intermedia del codigo de alto nivel.
La intención es generar tripletas y posterior a ello generar código 3 direcciones o código 3 direcciones ejecutable a partir de los cuartetos.



(a) Árbol sintáctico

	op	arg_1	arg_2
0	menos	c	
1	*	b	(0)
2	menos	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
	...		

(b) Triples